

2. 퍼셉트론

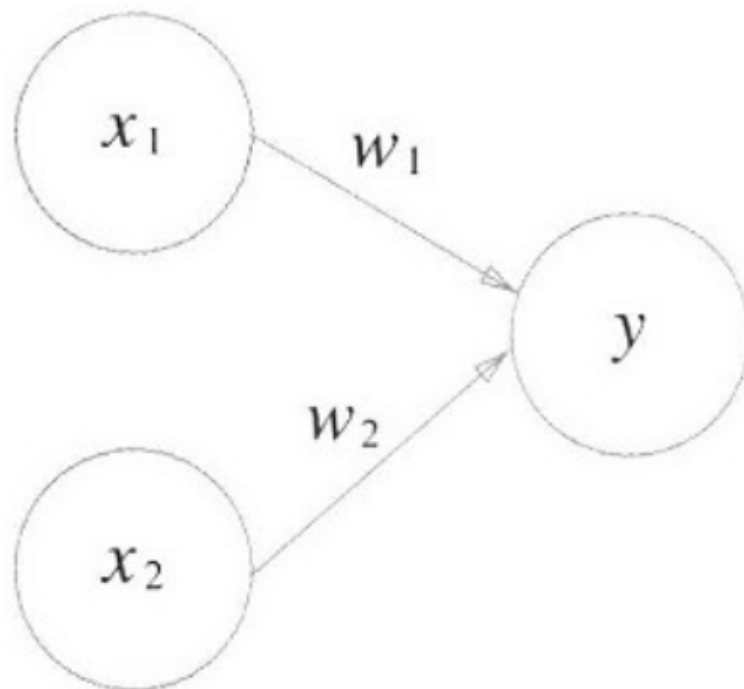
2.1 퍼셉트론이란?

퍼셉트론

-> 다수의 신호를 입력 받고, 하나의 신호를 출력하는 것(0 or 1)

그림의 원 -> 노드 OR 뉴런

신호 -> 하나의 "흐름"



$$y = \begin{cases} 0 & (w_1x_1 + w_2x_2 \leq \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$$

w -> 가중치 -> 각 신호가 얼마나 영향을 줄지를 정하는 요소
-> 당연히 가중치가 클수록 영향이 큰 신호가 됨

θ -> 임계값 -> 뉴런이 활성화되는 것에 대한 기준점

2.2 단순한 논리 회로

2.2.1 AND 게이트

그림 2-2 AND 게이트의 진리표

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

AND 게이트 -> 신호가 둘 다 1 이어야 뉴런이 **활성화되는** (출력이 1이되는) 논리 회로

if) **AND** 게이트를 구현하고 싶다

-> (w_1, w_2, θ) 값을 알맞게 정해주면 됨

-> 당연히 무수히 많음 -> 결국 매개변수 설정으로 게이트를 구현하는 것

-> 왜 w_1 , w_2 , θ 만 이용하나 -> x_1 , x_2 , y 는 출력값이 0, 1 로 한정되어 있음

2.2.2 NAND 게이트와 OR 게이트

NAND -> Not AND -> AND 의 반대

OR -> 입력 신호 중 하나 이상이 1이면 출력이 1이 되는 논리 회로

그림 2-3 NAND 게이트의 진리표

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

그림 2-4 OR 게이트의 진리표

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

NAND 게이트 구현 예시

-> (-0.5, -0.5, -0.7)

-> 사실 그냥 **AND** 게이트가 되는 매개변수에서 부호만 바꿔주면 됨 -> 당연한 것

OR 게이트 구현 예시

-> (2, 2, 1)

-> 이것도 드럽게 많음

// 결국 퍼셉트론 구조는 다 똑같음 -> 매개변수만 다를 뿐

+

여기서 퍼셉트론의 매개변수를 설정하는 것은 컴퓨터가 아니라 우리가 하는 작업이다

-> 인간이 직접 진리표라는 '학습 데이터'를 보면서 매개변수의 값을 생각해 봐야 한다

-> '기계학습문제'는 위의 작업을 컴퓨터가 자동으로 하게 하는 것

-> '학습'이란? -> 적절한 매개변수를 찾는 작업

-> 사람은 퍼셉트론의 구조(모델)를 고민하고 컴퓨터에 학습할 데이터를 주는 일을 하게 된다

2.3 퍼셉트론 구현하기

2.3.1 간단한 구현부터

```
def AND(x1, x2):  
    w1, w2, theta = 0.5, 0.5, 0.7  
    tmp = x1*w1 + x2*w2  
    if tmp <= theta:  
        return 0  
    elif tmp > theta:  
        return 1
```

→ 출력

```
AND(0, 0) -> 0 출력  
AND(1, 0) -> 0 출력  
AND(0, 1) -> 0 출력  
AND(1, 1) -> 1 출력
```

2.3.2 가중치와 편향 도입

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

→ 2.1 에서의 식에서 $\theta \rightarrow -b$ 로 치환하고 이항한 것 → 기호만 바뀌었을 뿐 똑같은 식

b -> 편향(bias)

다르게 말하면 퍼셉트론은

-> 입력 신호에 가중치를 곱한 값과 편향을 더해서 그 값이 0을 넘으면 1, 아니면 0을 출력하는 것
-> why) 왜 이렇게 하나 -> 아직 안 나옴
-> 2.3.1 에서 구현한 코드는 직관적으로 알기 쉽지만 앞으로를 위해 수정

바로 위에 식을 코드로 구현

```
import numpy as np
x = np.array([0, 1]) // 입력
w = np.array([0.5, 0.5]) // 가중치
b = -0.7 // 편향
w*x
// array([0, 0.5])
np.sum(w*x)
// 0.5
np.sum(w*x) + b
// -0.19999999999999996 대략 -0.2 (부동소수점에 의한 연산 오차)
```

-> numpy 배열끼리의 곱은 두 배열의 원소 수가 같다면 -> 각 원소끼리 곱함
-> nu.sum -> 모든 원소 총합을 계산 -> 여기에 편향(b)를 더하면 식 완성

2.3.3 가중치와 편향 구현하기

```
def AND(x1, x2):
    x = np.array([x1, x2])
    w = np.array([0.5, 0.5])
    b = -0.7
    tmp = np.sum(w*x) + b
    if tmp <= 0:
        return 0
    else:
        return 1
```

→ 가중치와 편향으로 AND 게이트 구현한 것

b(편향)은 그냥 **"-0"** 라는 사실은 기억해야 함 // 부호 헛갈리지 않기
 -> **w1, w2** 와는 기능이 다르다는 사실 유의해야 함
 -> **w1, w2** -> 신호가 결과에 주는 **영향력**(중요도)을 조절하는 매개변수
 -> **b** 뉴런이 얼마나 쉽게 **활성화**(결과로 **1**을 출력)하느냐를 조정하는 매개변수
 -> **b = -0.1** -> **sum(x*w)**가 **0.1**을 초과할 때만 활성화
 -> **b = -20** -> **sum(x*w)**가 **20**을 초과할 때만 활성화

NAND 게이트

```
def NAND(x1, x2):
    x = np.array([x1, x2])
    w = np.array([-0.5, -0.5])
    b = 0.7
    tmp = np.sum(x*w) + b
    if tmp <= 0:
        return 0
    else:
        return 1
```

OR 게이트

```
def OR(x1, x2):
    x = np.array([x1, x2])
    w = np.array([0.5, 0.5])
    b = -0.2
    tmp = np.sum(x*w) + b
    if tmp <= 0:
        return 0
    else:
        return 1
```

2.4 퍼셉트론의 한계

2.4.1 도전! XOR 게이트

XOR 게이트 -> "배타적 논리합"이라는 논리 회로 -> x_1 , x_2 중 한쪽만 1일 때 1 출력
// 쉽게 말하면 같으면 0 다르면 1 이런 느낌

그림 2-5 XOR 게이트의 진리표

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

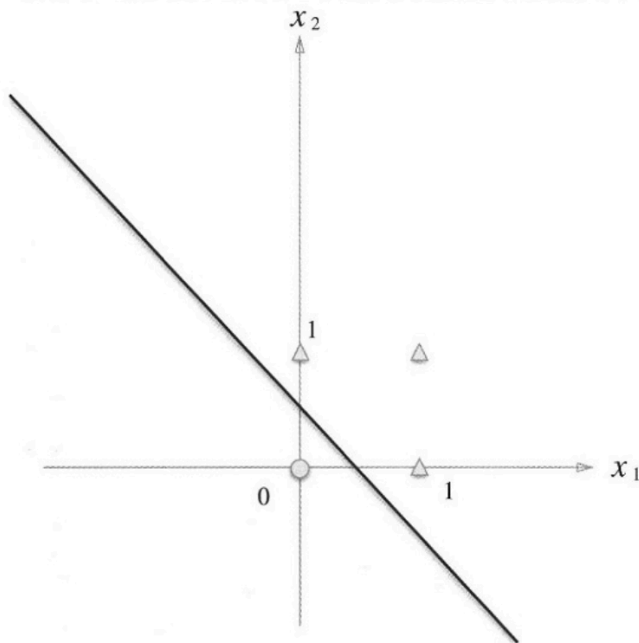
XOR 게이트는 우리가 알던 사실로는 퍼셉트론 구현이 안됨
-> 왜 why // 바로 아래 참고, 귀납적으로 대충 예측 가능

우선 OR 게이트의 동작을 시각적으로 생각해보죠. OR 게이트는, 예를 들어 가중치 매개변수가 $(b, w_1, w_2) = (-0.5, 1.0, 1.0)$ 일 때 [그림 2-4]의 진리표를 만족합니다. 이때의 퍼셉트론은 [식 2.3]으로 표현됩니다.

$$y = \begin{cases} 0 & (-0.5 + x_1 + x_2 \leq 0) \\ 1 & (-0.5 + x_1 + x_2 > 0) \end{cases} \quad [\text{식 2.3}]$$

[식 2.3]의 퍼셉트론은 직선으로 나뉜 두 영역을 만듭니다. 직선으로 나뉜 한쪽 영역은 1을 출력하고 다른 한쪽은 0을 출력합니다. 이를 그려보면 [그림 2-6]처럼 됩니다.

그림 2-6 퍼셉트론의 시각화 : 회색 영역은 0을 출력하는 영역이며, 전체 영역은 OR 게이트의 성질을 만족한다.



직선 하나로 '동그라미'와 '세모'가 명확히 나눌 수 있음

+ 왜 하필 직선이나

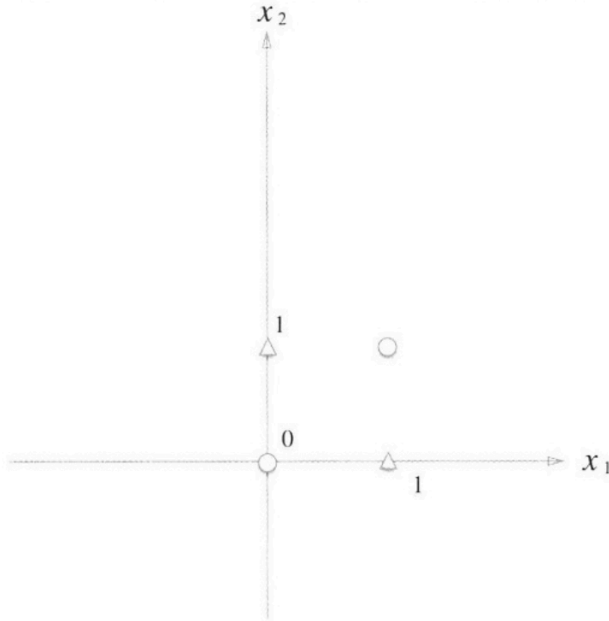
-> 식이 $b + x_1 * w_1 + x_2 * w_2$ 이 0보다 크냐 작냐 인데

-> y축 : x_2 , x축: x_1 , 상수: b

-> 직선의 방정식 만족

그럼 XOR 게이트의 경우는 어떨까요? OR 게이트 때처럼 직선 하나로 ○과 △을 나누는 영역을 만들어낼 수 있을까요?

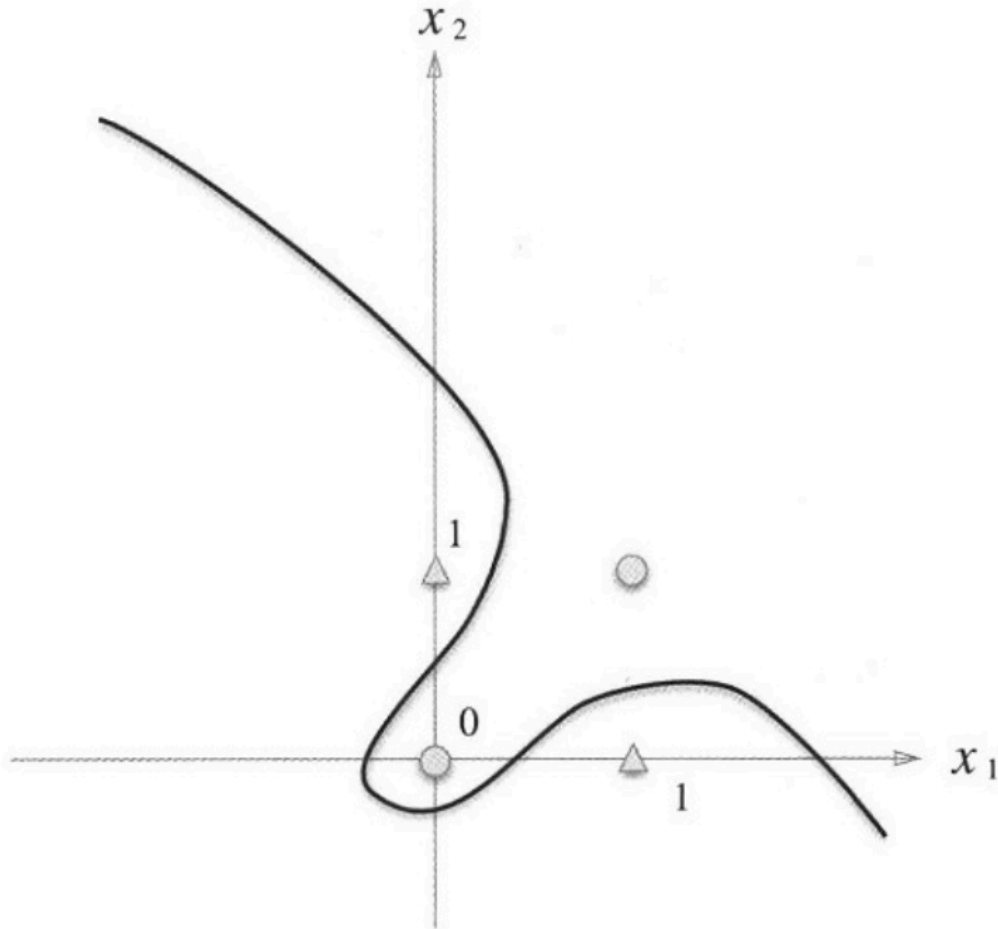
그림 2-7 ○과 △은 XOR 게이트의 출력을 나타낸다. 직선 하나로 ○과 △을 나누는 영역을 만들 수 있을까?



[그림 2-7]의 ○과 △을 직선 하나로 나누는 방법은 아무리 생각해도 떠오르지 않습니다. 사실 직선 하나로 나누기란 불가능합니다.

2.4.2 선형과 비선형

그림 2-8 곡선이라면 ○과 △을 나눌 수 있다.



퍼셉트론은 직선 하나로 나눌 수 있는 영역만 표현가능하다는 '명확한 한계'가 존재

선형 -> 직선의 영역

비선형 -> 곡선의 영역

2.5 다층 퍼셉트론이 출동한다면

XOR 게이트는 '하나의 퍼셉트론'으로는 표현 불가

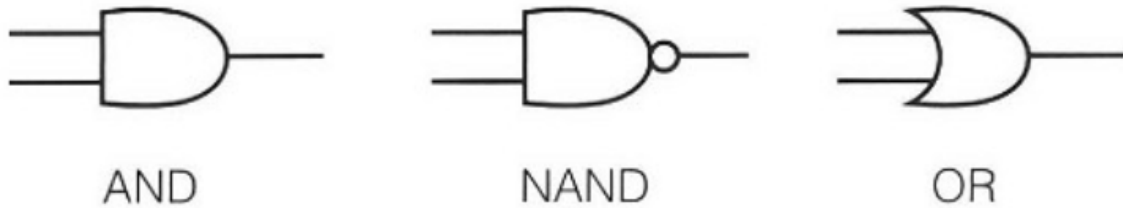
-> 그러면 '여러 개의 퍼셉트론'을 이용하면 된다

-> '퍼셉트론 층을 쌓는다'라고 표현

2.5.1 기존 게이트 조합하기

XOR 게이트를 만드는 방법은 다양함 → 그 중 하나는 AND, NAND, OR 게이트를 조합하는 방법

그림 2-9 AND, NAND, OR 게이트 기호



- NAND에 붙어있는 'o' = "출력을 반전한다" 라는 의미

[그림 2-11]과 같은 조합이라면 XOR 게이트를 구현할 수 있습니다. x_1 과 x_2 가 입력 신호, y 가 출력 신호입니다. x_1 과 x_2 는 NAND와 OR 게이트의 입력이 되고, NAND와 OR의 출력이 AND 게이트의 입력으로 이어집니다.

그림 2-11 AND, NAND, OR 게이트를 조합해 구현한 XOR 게이트

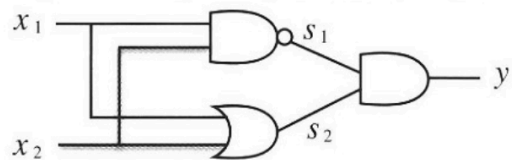


그림 [그림 2-11]의 조합이 정말 XOR를 구현하는지 살펴봅시다. NAND의 출력을 s_1 , OR의 출력을 s_2 로 해서 진리표를 만들면 [그림 2-12]처럼 됩니다. x_1 , x_2 , y 에 주목하면 분명히 XOR의 출력과 같습니다.

그림 2-12 XOR 게이트의 진리표

x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

2.5.2 XOR 게이트 구현하기

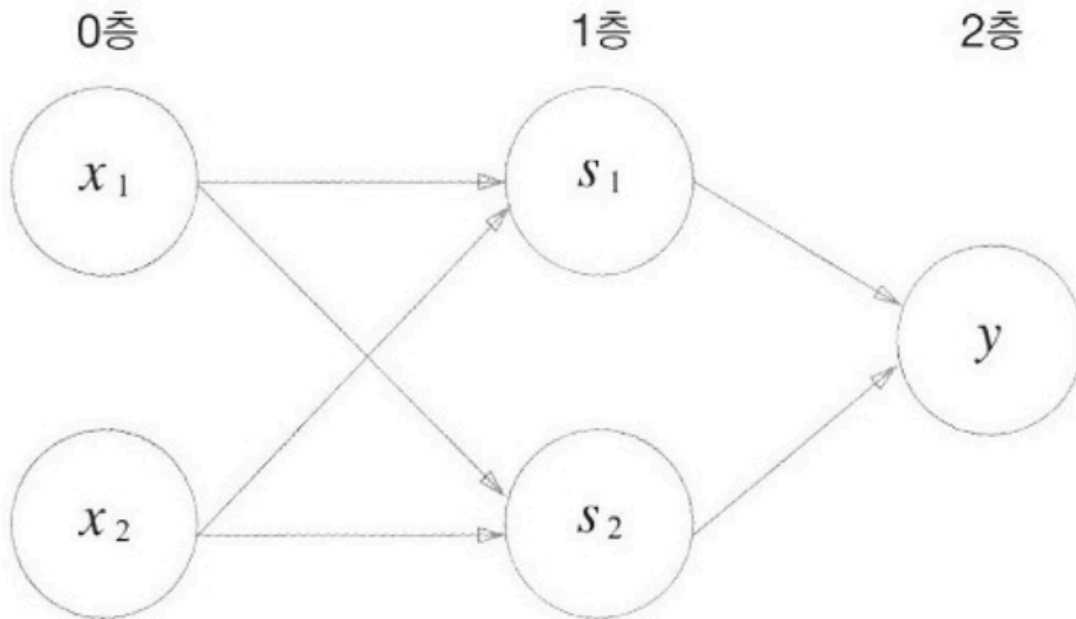
2.5.1 에서 했던 것처럼, AND NAND OR 게이트를 조합한다면 매우 쉽게 구현 가능하다

```
def XOR(x1, x2):
    s1 = NAND(x1, x2)
    s2 = OR(x1, x2)
    y = AND(s1, s2)
    return y
```

→ 출력

```
XOR(0, 0) // 0 출력
XOR(1, 0) // 1 출력
XOR(0, 1) // 1 출력
XOR(1, 1) // 0 출력
```

그림 2-13 XOR의 퍼셉트론



AND, OR이 단층 퍼셉트론인데 반해, XOR 게이트는 2층 퍼셉트론
 -> 이처럼 여러 개인 퍼셉트론 -> "다층 퍼셉트론"

위에 과정을 말로 풀면

1. 0층의 두 뉴런이 입력 신호를 받아 1층의 뉴런으로 신호를 보낸다
2. 1층의 뉴런이 2층의 뉴런으로 신호를 보내고, 2층의 뉴런은 y를 출력한다

-> 결국 다층 퍼셉트론으로 다양한 게이트를 구현할 수 있다

+ [그림 2-13] 은 모두 3층으로 구성되어있지만, 가중치를 갖는 층은 2개 뿐

-> '2층 퍼셉트론'이라고 부르기로 해보자 -> 문헌에 따라서는 3층 이라고 하기도 한다는 사실만 알아두자

2.6 NAND에서 컴퓨터까지

다층 퍼셉트론은 지금까지 보아온 회로보다 복잡한 회로를 만들 수 있습니다. 예를 들면, 덧셈을 처리하는 가산기도 만들 수 있습니다. 2진수를 10진수로 변환하는 인코더, 어떤 조건을 충족하면 1을 출력하는 회로(패리티 검사 회로)도 퍼셉트론으로 표현할 수 있습니다. 사실은 퍼셉트론을 이용하면 '컴퓨터'마저 표현할 수 있습니다!

컴퓨터는 정보를 처리하는 기계죠. 컴퓨터에 무언가를 입력하면 정해진 방법으로 처리하고 그 결과를 출력합니다. 정해진 방법으로 처리한다는 것은 컴퓨터도 마치 퍼셉트론처럼 입력과 출력으로 구성된 특정 규칙대로 계산을 수행한다는 뜻입니다.

컴퓨터 내부에서 이뤄지는 처리가 매우 복잡할 거 같지만, 사실은 (놀랍게도) NAND 게이트의 조합만으로 컴퓨터가 수행하는 일을 재현할 수 있습니다. NAND 게이트만으로 컴퓨터를 만들 수 있다? 이 말은 곧 퍼셉트론으로도 컴퓨터를 표현할 수 있다는 놀라운 사실로 이어집니다. 지금까지 살펴본 것처럼 NAND 게이트는 퍼셉트론으로 만들 수 있기 때문이죠.

이처럼 다층 퍼셉트론은 컴퓨터도 만들 정도로 복잡한 표현을 해냅니다. 대견하죠! 그럼 어떤 구조의 퍼셉트론이면 컴퓨터를 표현할 수 있을까요? 층을 얼마나 깊게 하면 컴퓨터가 만들어질까요?

그 답은 "이론상 2층 퍼셉트론이면 컴퓨터를 만들 수 있다"입니다. 말도 안 되는 소리 같지만, 2층 퍼셉트론, 정확히는 비선형인 시그모이드 함수를 활성화 함수로 이용하면 임의의 함수를 표현할 수 있다는 사실이 증명되었습니다(3장 참고). 그러나 2층 퍼셉트론 구조에서 가중치를 적절히 설정하여 컴퓨터를 만들기란 너무 어렵습니다. 실제로도 NAND 등의 저수준 소자에서 시작하여 컴퓨터를 만드는 데 필요한 부품(모듈)을 단계적으로 만들어가는 쪽이 자연스러운 방법입니다. 즉, 처음에는 AND와 OR 게이트, 그다음에는 반가산기와 전가산기, 그다음에는

산술 논리 연산 장치(ALU), 그다음에는 CPU라는 식이죠. 그래서 퍼셉트론으로 표현하는 컴퓨터도 여러 층을 다시 층층이 겹친 구조로 만드는 방향이 자연스러운 흐름입니다.

이 책에서는 컴퓨터를 만들지 않습니다. 그래도 퍼셉트론은 층을 거듭 쌓으면 비선형적인 표현도 가능하고, 이론상 컴퓨터가 수행하는 처리도 모두 표현할 수 있다는 점을 기억해주세요.

결국 퍼셉트론들의 조합만으로
-> "컴퓨터"까지 구현이 가능하다

+ 시그모이드 함수 -> 어떤 실수든 0과 1사이의 값으로 부드럽게 눌러담는 함수

EX) $h(x) = 1 / (1 + \exp(-x))$

-> 값을 직접 넣어보면

$x = -5 \rightarrow 1/(1+e^5) \approx 0.007$ (거의 0)

$x = -1 \rightarrow 1/(1+e^1) \approx 0.269$

$x = 0 \rightarrow 1/(1+1) = 0.5$

$x = 1 \rightarrow 1/(1+e^{-1}) \approx 0.731$

$x = 5 \rightarrow 1/(1+e^{-5}) \approx 0.993$ (거의 1)

step 함수와 비교

	step	sigmoid
출력	0 또는 1 (둘 중 하나)	0~1 사이 연속값
경계(x=0)	뚝 끊김	부드럽게 변함
미분	거의 모든 곳에서 0	모든 곳에서 0이 아님

2.7 추가적인 궁금증

2.7.1 선형으로 해결되지 않는다는 사실에 대한 증명1(수식)

부등식모순 (귀류법)

XOR을 구현하는 퍼셉트론 $y = 1$ if $w_1*x_1 + w_2*x_2 + b > 0$, else 0
이 존재한다고 가정

-> 그러면 4개 입력에 대해 다음이 모두 성립

$(0, 0) \rightarrow 0 : b \leq 0 \dots (1)$

$(1, 0) \rightarrow 1 : w_1 + b > 0 \dots (2)$

$(0, 1) \rightarrow 1 : w_2 + b > 0 \dots (3)$

$(1, 1) \rightarrow 0 : w_1 + w_2 + b \leq 0 \dots (4)$

1. (2)+(3)

-> $w_1 + w_2 + 2b > 0$

2. (1)에서 $b \leq 0$ 이므로 $-b \geq 0$ 양변에서 b를 빼면:

-> $w_1 + w_2 + b > -b \geq 0$

3. 즉, $w_1 + w_2 + b > 0$

-> (4)와 모순

-> 따라서 그런 w_1, w_2, b 는 존재하지 않음

-> 가정이 틀렸으니 어떤 직선(어떤 가중치·편향)으로도 XOR은 불가능

-> 모든 가능한 직선을 다 따졌으니, 귀납이 아니라 연역

2.7.2 선형으로 해결되지 않는다는 사실에 대한 증명2(기하)

불록깍질(기하학)

-> 선분의 중점은 양 끝점을 평균한 점이라는 성질을 이용합니다.

1. 선형 함수 $f(x) = w_1 \cdot x_1 + w_2 \cdot x_2 + b$ 는 중점에서 $f(\text{중점}) = (f(A) + f(B))/2$ 를 만족

2. Δ 두 점 $(1, 0), (0, 1)$ 의 중점은 $(0.5, 0.5)$, $\circ$ 두 점 $(0, 0), (1, 1)$ 의 중점도

-> 똑같이 $(0.5, 0.5)$

3. 직선이 Δ 둘을 양의 쪽에 두려면 $f(1, 0) > 0, f(0, 1) > 0$ 이므로

-> $f(0.5, 0.5) > 0$ 이어야 함

4. 직선이 $\circ$ 둘을 양의 쪽에 두려면 $f(0, 0) \leq 0, f(1, 1) \leq 0$ 이므로

-> $f(0.5, 0.5) \leq 0$ 이어야 함

5. 같은 점에서 f 가 동시에 >0 이고 $\leq 0 \rightarrow$ 모순

2.7.3 XOR 게이트를 식으로 표현하기

그냥 단순히 $w_1x_1 + w_2x_2 + b > 0$ 를 3번 사용하면 된다

단위함수 // 위에 나온 시그모이드 함수 중 거친 버전 느낌

$\text{step}(z) = 1 \text{ if } z > 0 \text{ else } 0$

-> 퍼셉트론 하나를 사용

1층

$s_1 = \text{step}(-0.5 \cdot x_1 + -0.5 \cdot x_2 + 0.7) \leftarrow \text{NAND}$

```
s2 = step( 0.5*x1 + 0.5*x2 - 0.2) ← OR
```

2층

```
y = step(0.5*s1 + 0.5*s2 - 0.7) ← AND
```

한줄로 합치면

```
y = step( 0.5*step(-0.5*x1 - 0.5*x2 + 0.7)  
          + 0.5*step( 0.5*x1 + 0.5*x2 - 0.2) - 0.7 )
```

→ 검증(1, 0)

```
s1 = step(-0.5 + 0.7) = step(0.2) = 1  
s2 = step( 0.5 - 0.2) = step(0.3) = 1  
y  = step(0.5 + 0.5 - 0.7) = step(0.3) = 1 ✓
```

→ 검증(1, 1)

```
s1 = step(-1 + 0.7) = step(-0.3) = 0  
s2 = step( 1 - 0.2) = step(0.8) = 1  
y  = step(0 + 0.5 - 0.7) = step(-0.2) = 0 ✓
```

$w_1x_1 + w_2x_2 + b > 0$ 이런 식 하나로는 구현 불가능

→ 하나로 된다는게 사실 그냥 퍼셉트론 하나만 쓰겠다는 것과 동치이기 때문